Systém pro správu skladu elektronických součástek

WMS – Elektronické součástky

Projektová dokumentace

Autoři: Tomáš Simandl
Tomáš Szetei
Lukáš Kvapil

Datum: 4. dubna 2026

Dokumentace systému pro skladovou evidenci elektronických součástek

Obsah

1 Datový model **2**

1.1 Formulace problému 2
1.2 Datový slovník 2
1.3 Diagram tříd 4
1.4 Stavový diagram 6
1.5 Activity diagram – Vytvoření pohybu 8
1.6 Sekvenční diagram – Vytvoření pohybu 9
1.7 Use Case 10
1.8 Přehled entit 12
1.9 Diagram tříd 12
1.10 Stavový diagram třídy `Pohyb` 14
1.11 Stavový diagram `Soucastka` (v projektu) 15
1.12 Use Case Diagram 16
1.13 Activity Diagram – Vytvoření pohybu 17
1.14 Sekvenční diagram – Vytvoření pohybu 18
1.15 Scénáře use case 18

Kapitola 1

Datový model

1.1 Formulace problému

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi.

1.2 Datový slovník

Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě.
- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.
- **Pohyb**: Abstraktní třída reprezentující pohyb součástek – příjem nebo výdej.
- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele.
- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu.
- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

Atributy

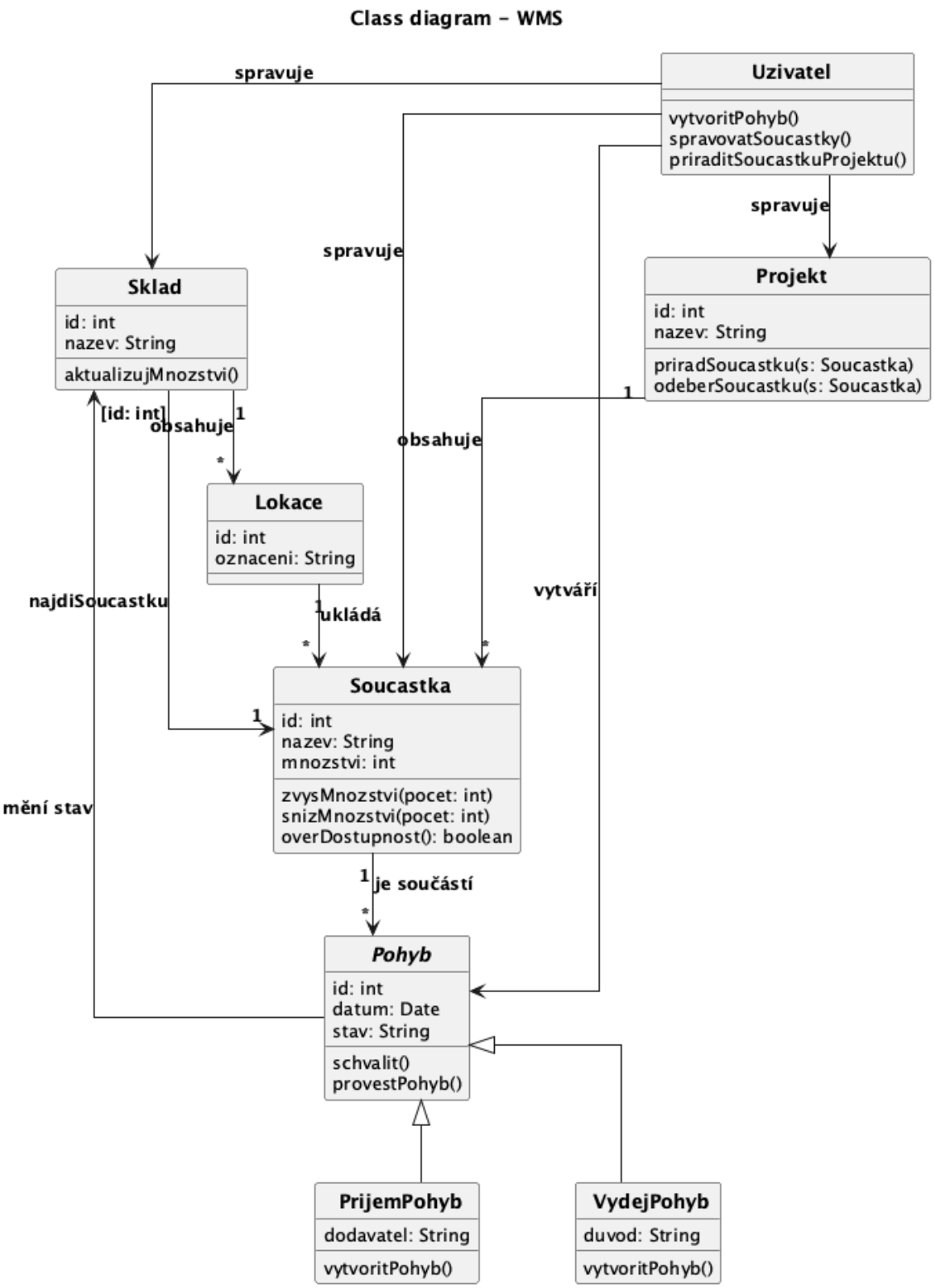
Tabulka 1.1: Přehled atributů tříd

Entita	**Atributy**
Soucastka	id, nazev, mnozstvi
Projekt	id, nazev
Pohyb	id, datum, stav
PrijemPohyb	dodavatel (dědí z Pohyb)
VydejPohyb	duvod (dědí z Pohyb)
Sklad	id, nazev
Lokace	id, oznaceni
Uzivatel	(správce systému, bez persistentních atributů)

1.3 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému – může vytvářet skladové pohyby, spravovat součástky, přiřazovat součástky k projektům a zobrazovat stav skladu.

Diagram využívá **generalizaci**: abstraktní třída `Pohyb` je základem pro dvě konkrétní podtřídy `PrijemPohyb` (naskladnění) a `VydejPohyb` (vyskladnění), které dědí společné atributy a přidávají vlastní specifické atributy. Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.

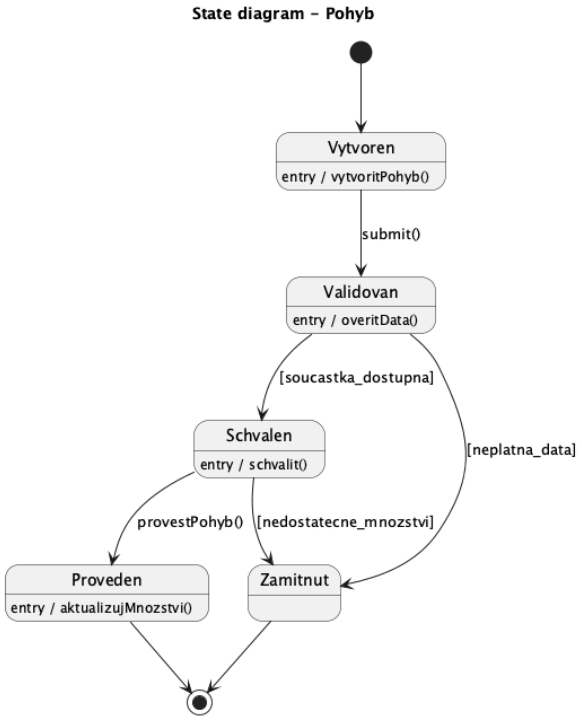


Obrázek 1.1: Class diagram WMS

1.4 Stavový diagram

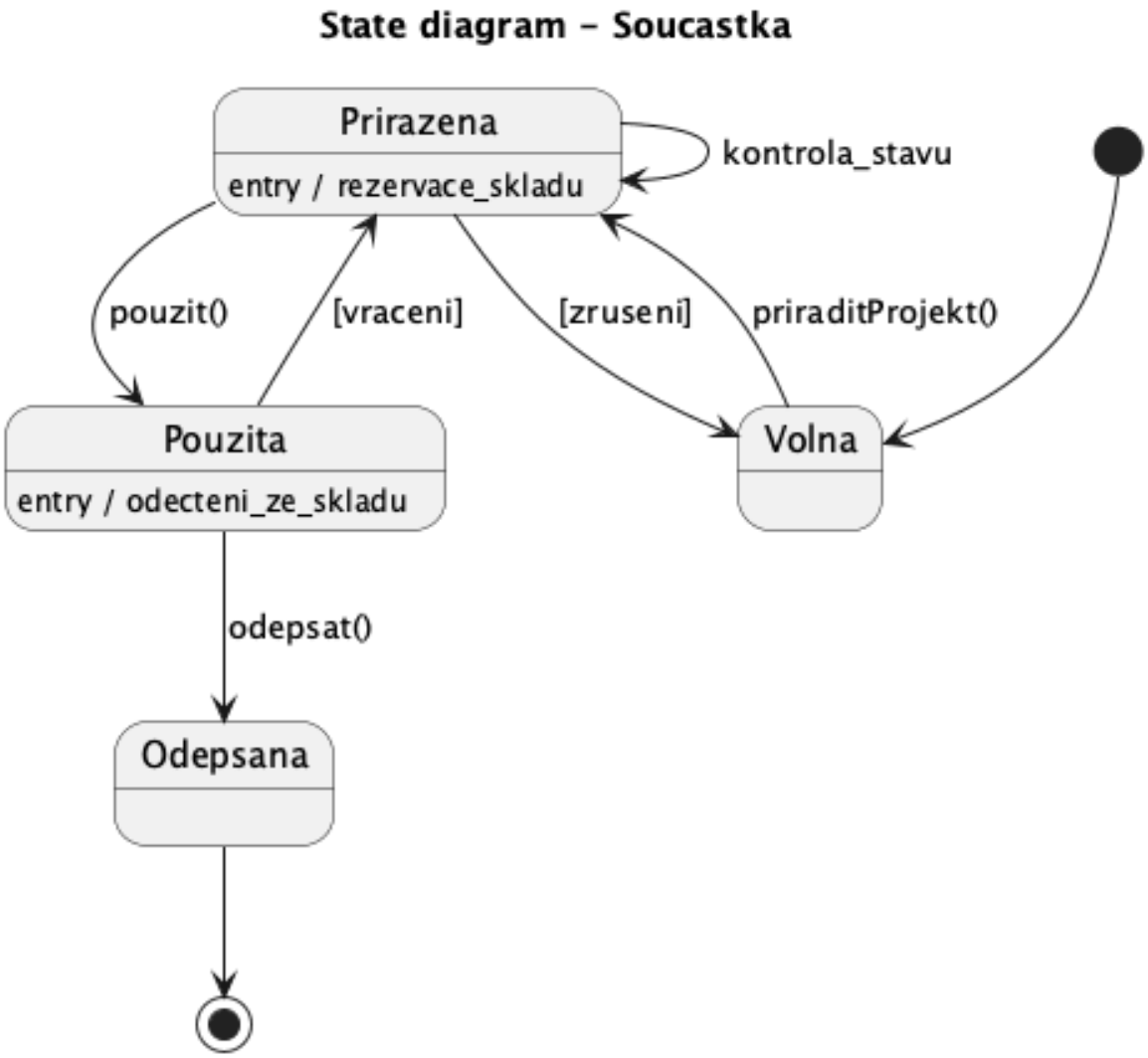
Stavový diagram třídy `Pohyb`

Třída `Pohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem: po vytvoření je odeslán k validaci, při které systém ověří, zda zadaná součástka existuje a je dostupná na skladě. Pohyb může být zamítnut při neplatných datech nebo při nedostatečném množství na skladě.



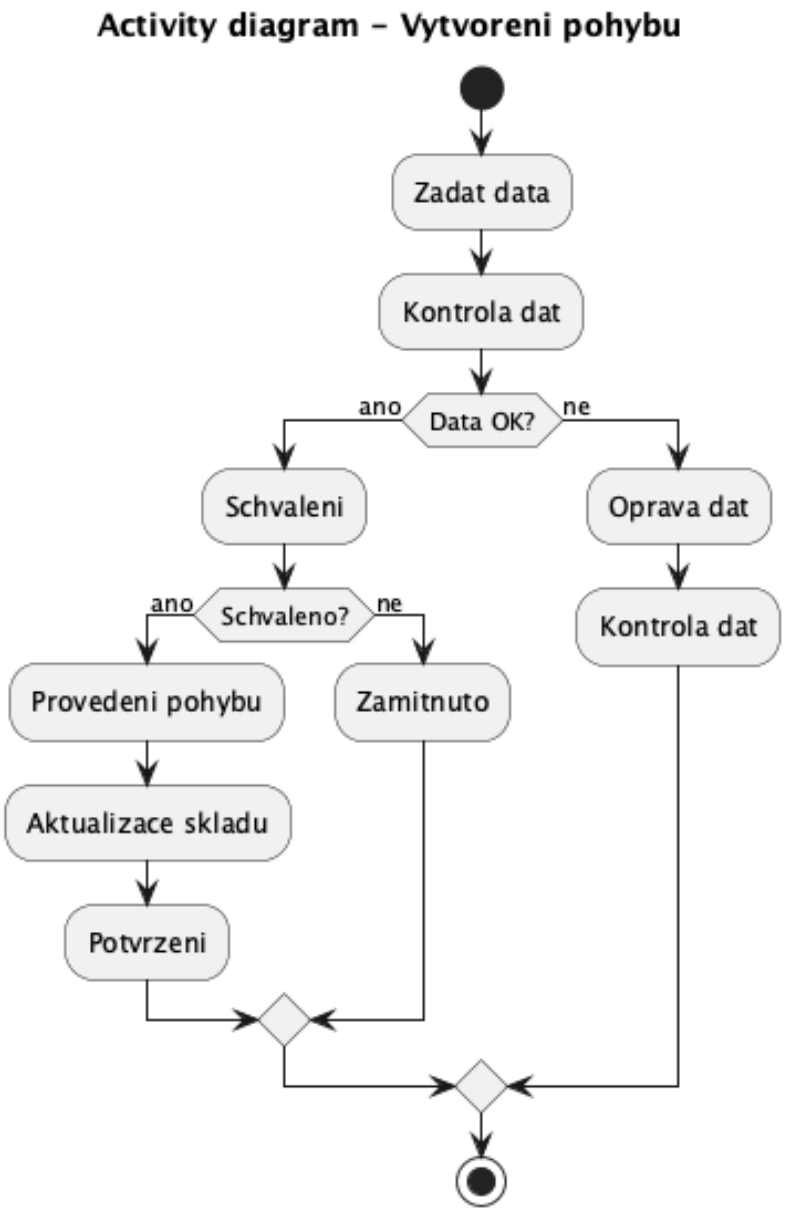
Obrázek 1.2: Stavový diagram třídy Pohyb

Stavový diagram `Soucastka` (v projektu)



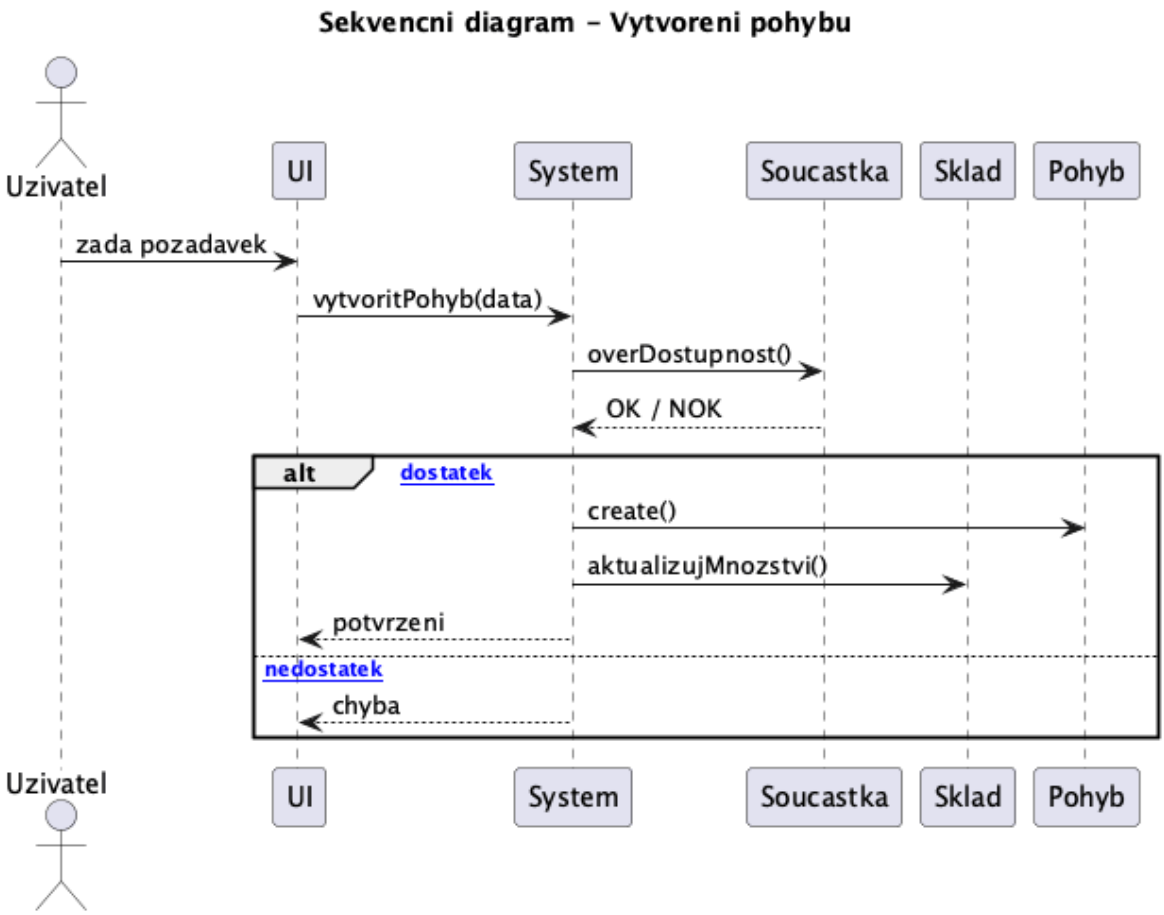
Obrázek 1.3: Stavový diagram třídy Soucastka (v projektu)

1.5 Activity diagram – Vytvoření pohybu



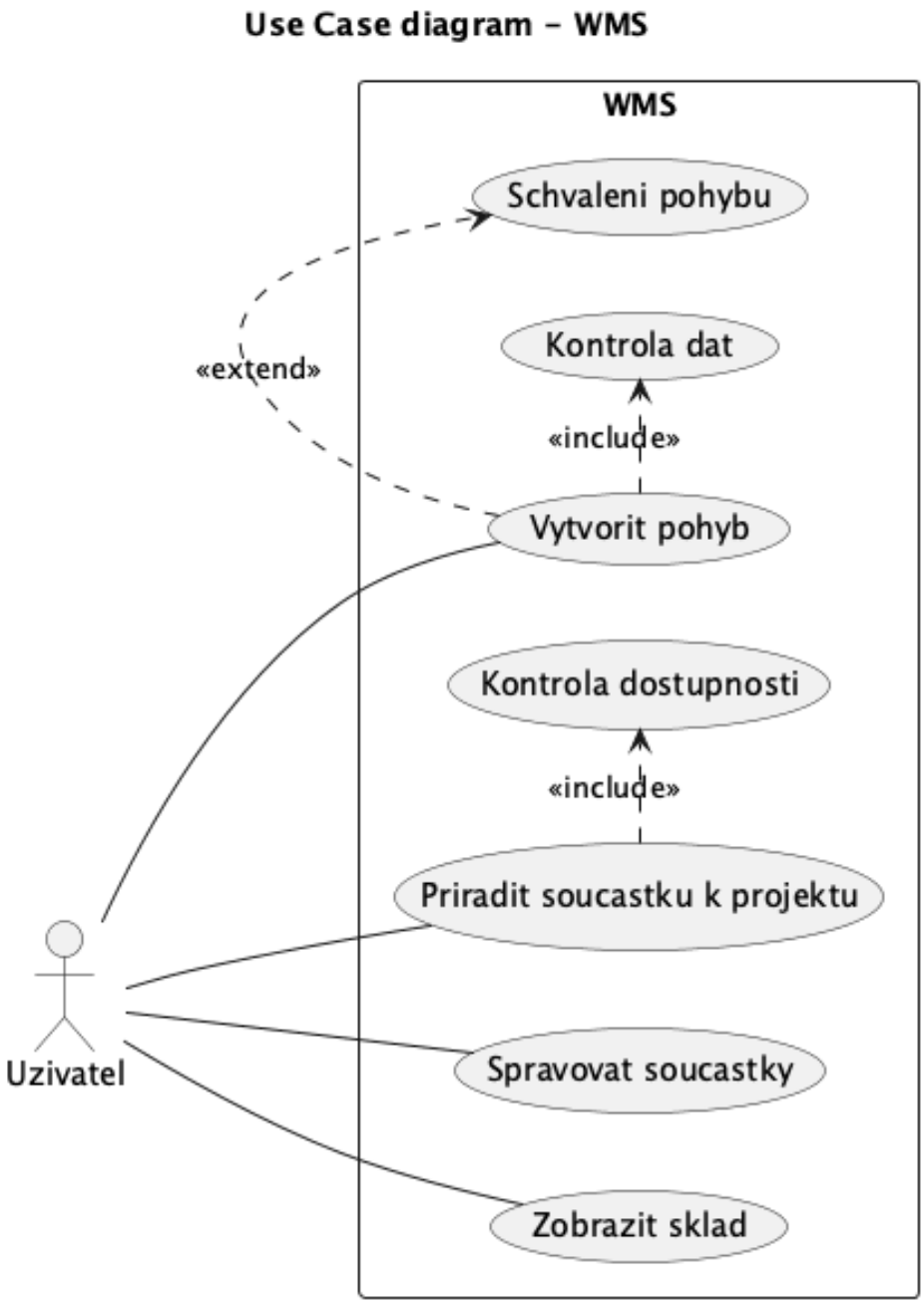
Obrázek 1.4: Activity diagram – Vytvoření pohybu

1.6 Sekvenční diagram – Vytvoření pohybu



Obrázek 1.5: Sekvenční diagram – Vytvoření pohybu

1.7 Use Case



Obrázek 1.6: Use Case Diagram – WMS

Scénář: Vytvoření pohybu

1. Uživatel zvolí *Vytvořit pohyb*.
2. Zadá součástku a množství.
3. Systém provede kontrolu dat.
4. Pokud `[data_ok]` → pohyb je schválen.

5. Systém provede pohyb a aktualizuje stav skladu.
6. Zobrazí se potvrzení výsledku.

Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt.
2. Zvolí součástku a zadá množství.
3. Systém ověří dostupnost voláním `overDostupnost()`.
4. Pokud `[dostatek]` → součástka je přiřazena.
5. Systém aktualizuje množství na skladě.

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi. Diagram využívá vybrané vlastnosti UML, jako jsou zobecnění, kardinality, agregace, kvalifikované vazby a atributy spojení, aby byl model systému co nejpřesnější.

1.8 Přehled entit

Systém pracuje s následujícími hlavními třídami:

`Uzivatel` Hlavní uživatel systému; vytváří pohyby a spravuje součástky.

`Sklad` Fyzický sklad identifikovaný názvem; sdružuje skladové lokace.

`Lokace` Konkrétní místo v skladu (přihrádka, regál) identifikované označením.

`Soucastka` Elektronická součástka evidovaná v systému; obsahuje atributy `id`, `nazev` a aktuální `mnozstvi` na skladě.

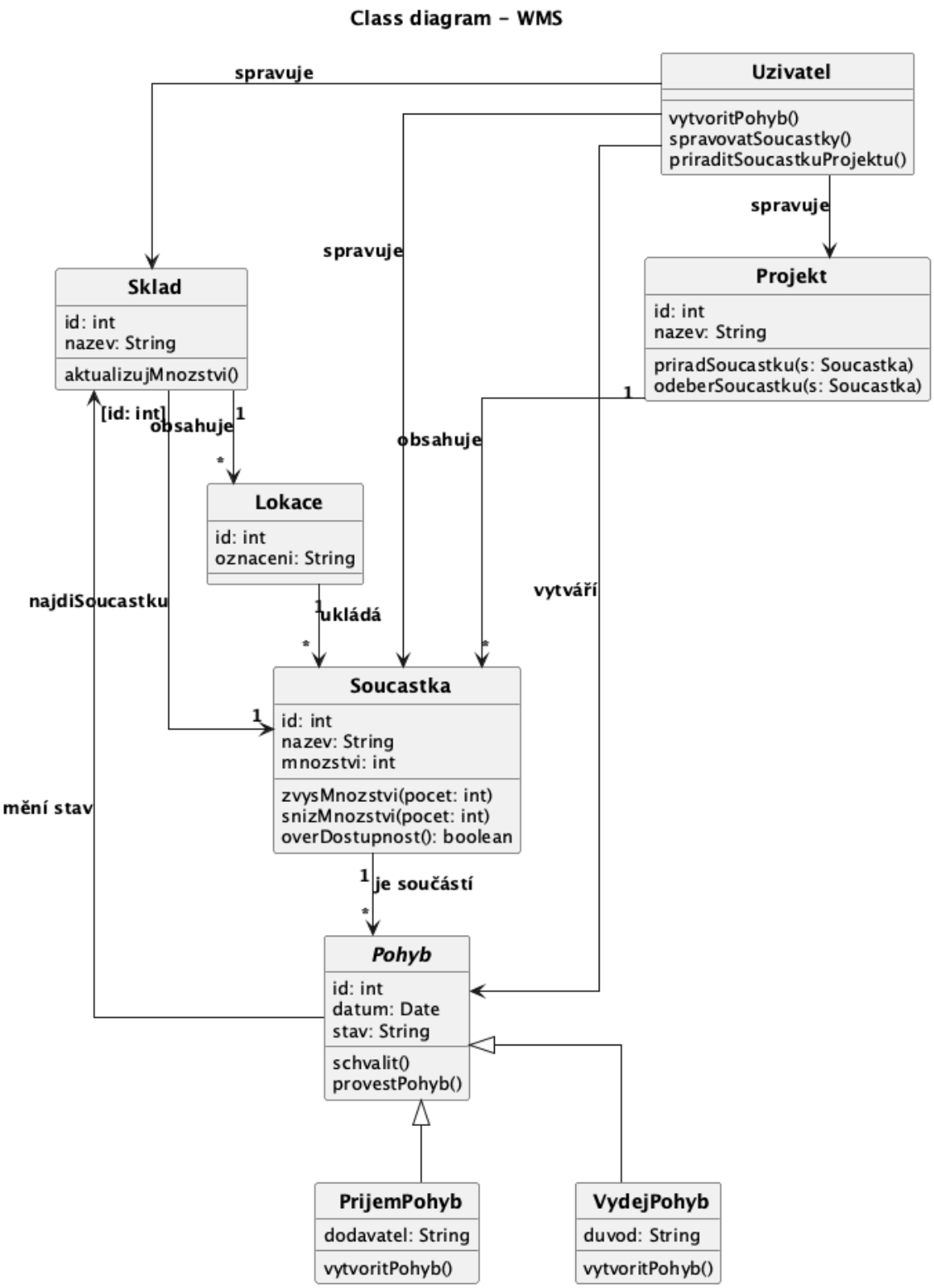
`Projekt` Projekt, ke kterému jsou součástky přiřazovány; identifikován pomocí `id` a `nazev`.

`Pohyb` Záznam o pohybu součástky (příjem, výdej, přesun); obsahuje `id`, `typ` pohybu, `datum` provedení a aktuální `stav`.

1.9 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému – může vytvářet skladové pohyby, spravovat součástky, přiřazovat součástky k projektům a zobrazovat stav skladu.

Diagram využívá **generalizaci**: abstraktní třída `Pohyb` je základem pro dvě konkrétní podtřídy `PrijemPohyb` (naskladnění) a `VydejPohyb` (vyskladnění), které dědí společné atributy a přidávají vlastní specifické atributy. Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.

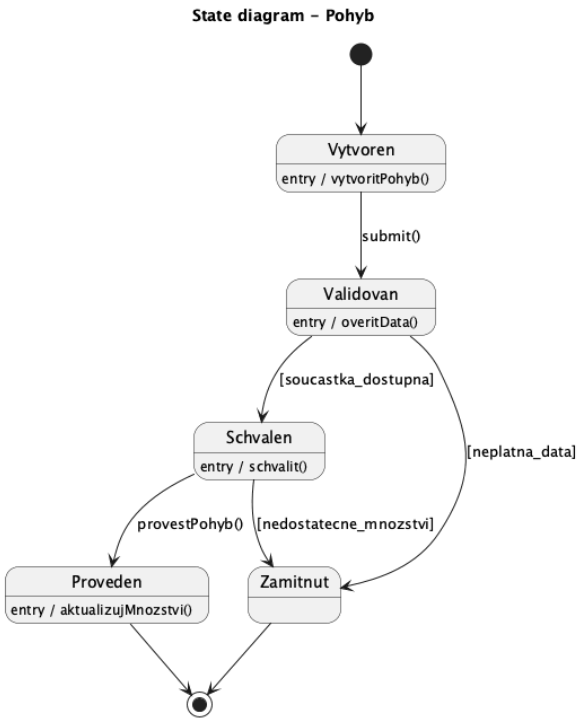


Obrázek 1.7: Class diagram WMS

1.10 Stavový diagram třídy `Pohyb`

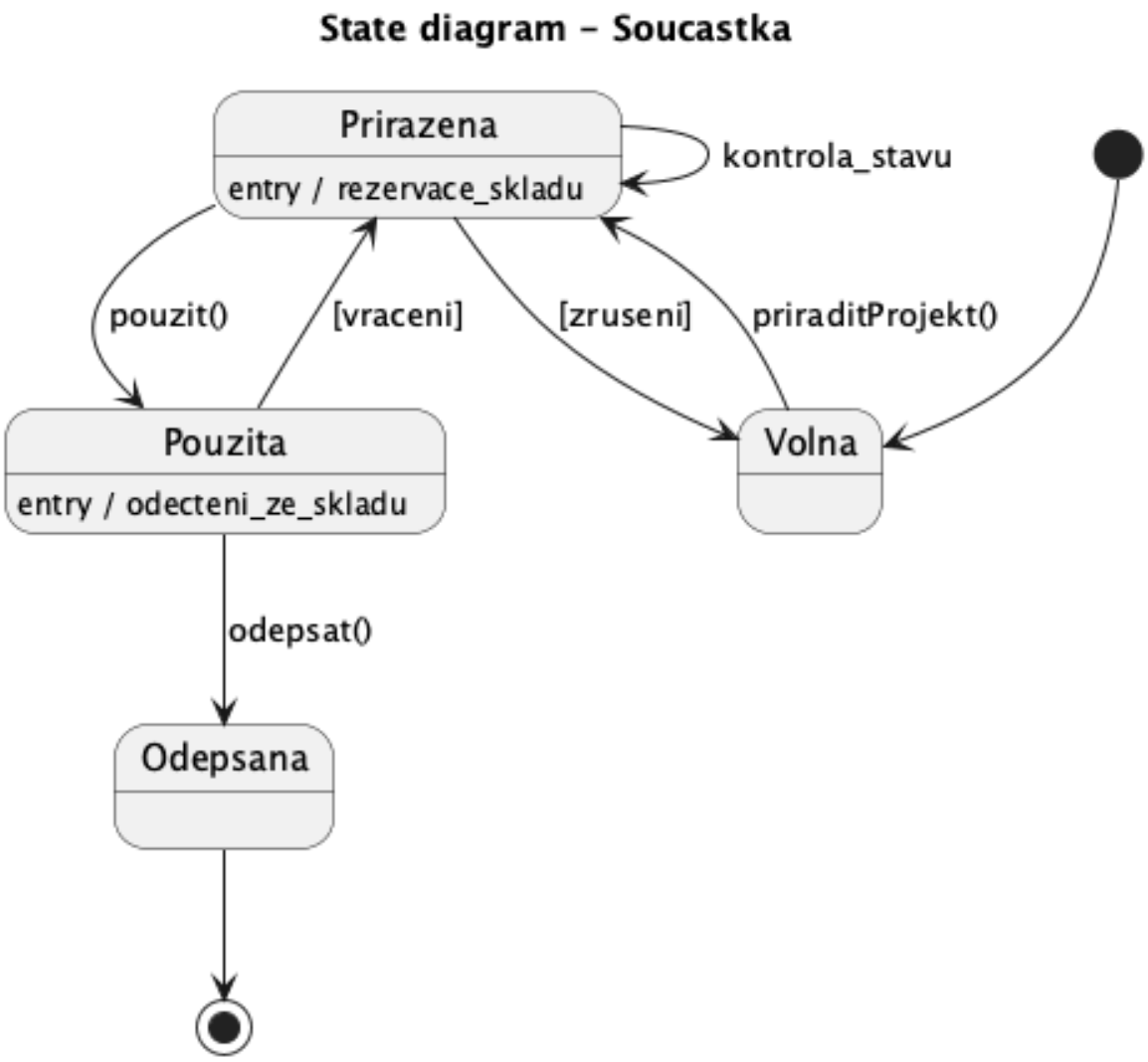
Třída `Pohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Atribut `typ` rozlišuje tři druhy pohybu: *příjem* (naskladnění nových součástek), *výdej* (odebrání součástek ze skladu) a *přesun* (přemístění součástek mezi lokacemi či sklady). Každý pohyb prochází životním cyklem: po vytvoření je odeslán k validaci (*Validován*), při které systém ověří, zda zadaná součástka existuje a je dostupná na skladě. Pokud jsou data platná, pohyb přejde do stavu *Schválen* a může být fyzicky proveden – dojde k aktualizaci množství na skladě. Pohyb může být zamítnut ve dvou situacích:

- při validaci (`[neplatna_data]`) – součástka s daným ID neexistuje v systému, chybí povinné údaje (typ pohybu, cílová lokace, datum) nebo zadané množství je 0 či záporné;
- po schválení (`[nedostatecne_mnozstvi]`) – požadované množství výdeje je vyšší než aktuální stav součástky na skladě.



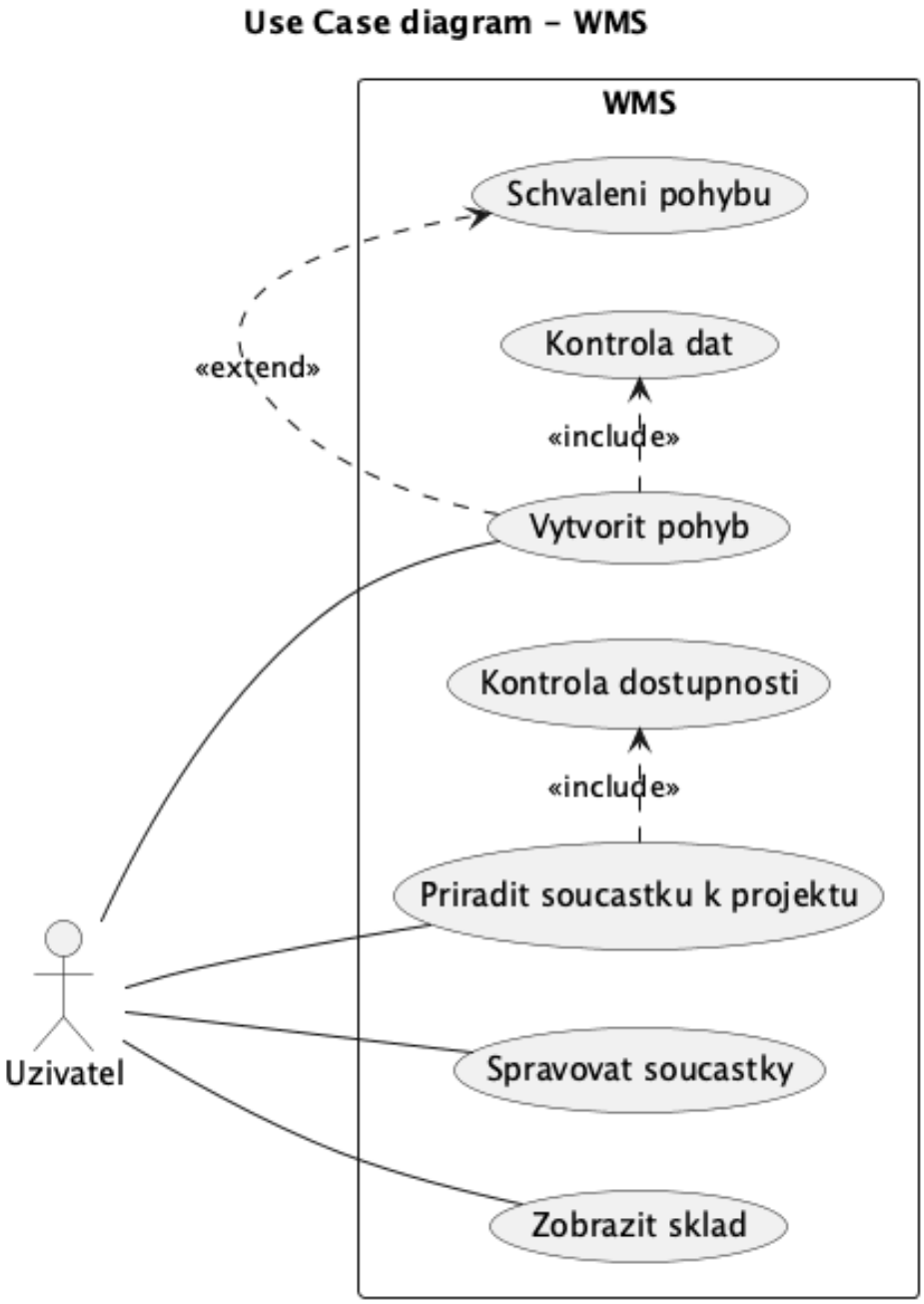
Obrázek 1.8: Stavový diagram třídy Pohyb

1.11 Stavový diagram `Soucastka` (v projektu)



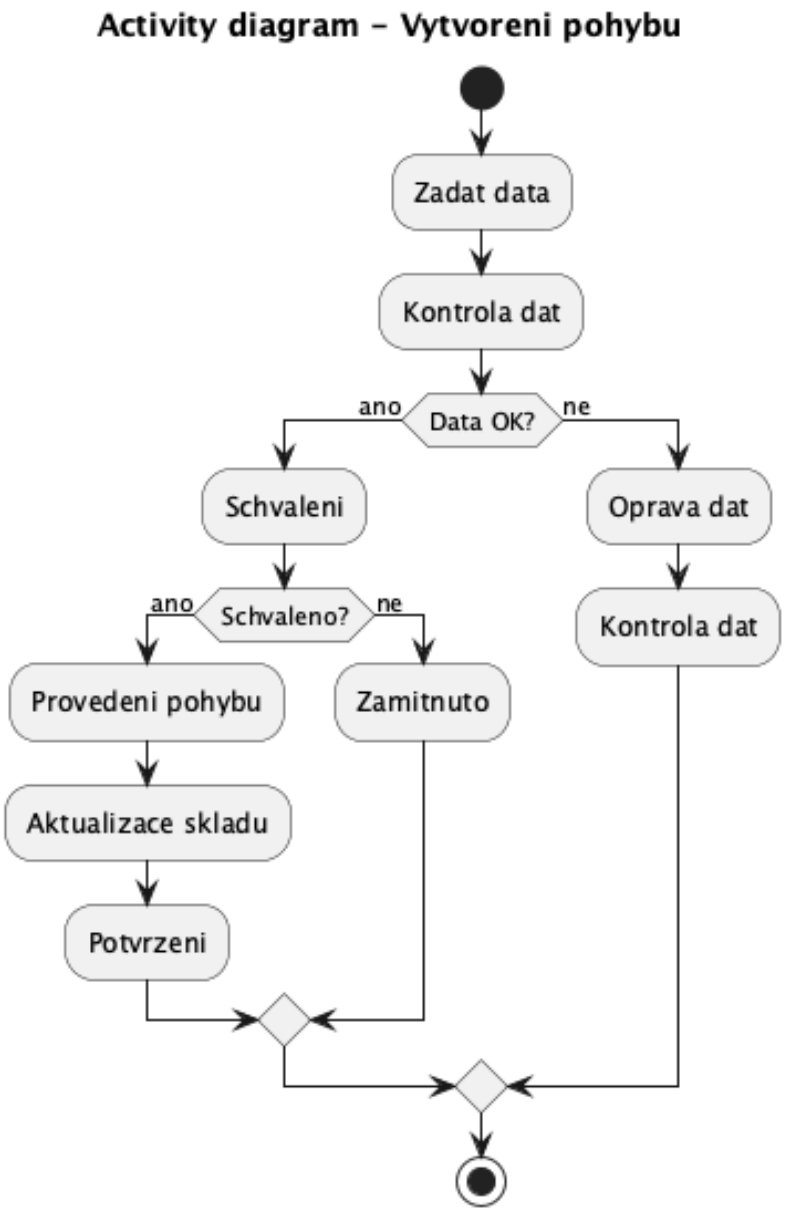
Obrázek 1.9: Stavový diagram třídy Soucastka (v projektu)

1.12 Use Case Diagram



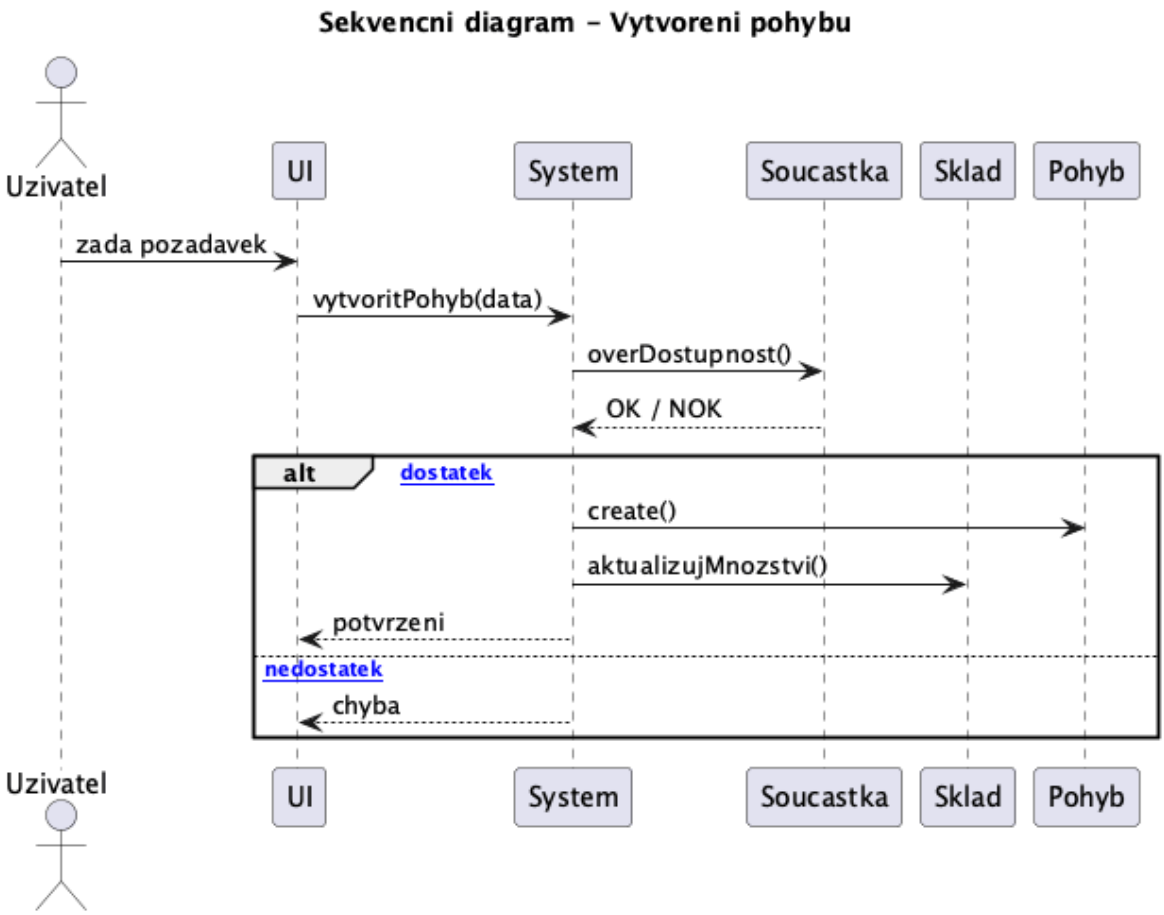
Obrázek 1.10: Use Case Diagram – WMS

1.13 Activity Diagram – Vytvoření pohybu



Obrázek 1.11: Activity diagram – Vytvoření pohybu

1.14 Sekvenční diagram – Vytvoření pohybu



Obrázek 1.12: Sekvenční diagram – Vytvoření pohybu

1.15 Scénáře use case

Scénář: Vytvoření pohybu

1. Uživatel zvolí *Vytvořit pohyb*.
2. Zadá součástku a množství.
3. Systém provede kontrolu dat.
4. Pokud `[data_ok]` → pohyb je schválen.
5. Systém provede pohyb a aktualizuje stav skladu.
6. Zobrazí se potvrzení výsledku.

Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt.
2. Zvolí součástku a zadá množství.
3. Systém ověří dostupnost voláním `overDostupnost()`.
4. Pokud `[dostatek]` → součástka je přiřazena.
5. Systém aktualizuje množství na skladě.